

ISM3232 — Week 15 Lab

Streamlit Business Interface — Instructor Facilitation Guide

ISM3232 · Business Application Development · USF Muma College of Business

Module 15 · Unit 4 · Capstone Build

Contents

1 Session Snapshot	2
2 Learning Objectives	2
3 Before Class — Setup Checklist	3
4 Materials Needed	3
5 Timing Plan (75 minutes, lab portion only)	3
6 Segment-by-Segment Walkthrough	5
6.1 Intro (0:00–0:04)	5
6.2 Part 1 — Install Streamlit and First Run (0:04–0:12, 8 min)	5
6.3 Part 2 — Build the Complete app.py (0:12–0:36, 24 min)	7
6.4 Part 3 — Test Every Feature (0:36–0:52, 16 min)	13
6.5 Part 4 — Adapt to Your Capstone Domain (0:52–1:02, 10 min)	14
6.6 Part 5 — Ritual and Push (1:02–1:08, 6 min)	15
6.7 Stretch — Bar Chart in the Report Tab (1:08–1:12, as time allows)	16
7 Wrap-Up (last ~3 minutes)	17
8 Appendix A — Full Answer Key (app.py)	17
9 Appendix B — Extra Practice (only if the class finishes early)	19

1 Session Snapshot

Course	ISM3232 — Business Application Development
Session	Week 15 Lab — Streamlit Business Interface
Unit	Unit 4 · Capstone Build
Class length	Full class period (75 minutes) for the lab; this week's session also includes a separate 60–90 minute FastAPI/REST API concept lecture with no lab deliverable — awareness only, budgeted outside this guide's 75-minute plan
Prerequisites	Week 14: complete, tested <code>database.py</code> (all five functions, ? placeholders, <code>db_file</code> parameters)
Student-facing lab page	Week 15 In-Class Lab — Module 7D, “Streamlit Business Interface”
Parts covered	Part 1 (install + first run) – Part 5 (test + ritual)
Submission	3 screenshots, GitHub URL, Canvas, completion credit

This is the lab where the capstone becomes a real, clickable application — a genuine, if simple, business tool a non-programmer could actually use. Every one of today's five tabs calls directly into Week 14's `database.py` functions; nothing about the data layer changes today, only a real interface sits in front of it. **A version-specific caution worth knowing before class:** this guide's code (matching the lab page) uses `st.dataframe(..., use_container_width=True)`, a parameter Streamlit has deprecated in favor of `width='stretch'`; depending on exactly which Streamlit version `pip install streamlit` pulls for your section, this may show a harmless deprecation warning or, in a sufficiently new version, may already require the updated syntax — verified below, with the fix ready if needed.

2 Learning Objectives

By the end of this class period, students should be able to:

1. Install and run a Streamlit app, understanding that it re-executes the entire script top to bottom on every interaction.
2. Build a multi-tab interface with `st.tabs()`, and use `st.text_input`, `st.selectbox`, `st.number_input`, and `st.button` to collect user input.
3. Wire every UI action directly to a `database.py` function — submit calls `add_record`, view/filter call `get_all_records`, update calls `update_status`, report calls `get_status_report`.

4. Test a full user workflow end to end through the running interface, not just by reading the code.
5. Adapt a generic five-tab template to their own capstone's specific domain, categories, and status values.

3 Before Class — Setup Checklist

- ☐ Run this guide's Part 2 `app.py` yourself before class and note which Streamlit version you have (`streamlit version` at the terminal) — if you see the `use_container_width` deprecation warning described above, decide whether to mention it in passing or actively demonstrate the `width='stretch'` replacement; if your version has fully removed the old parameter, switch your live-coded version to `width='stretch'` from the start rather than hitting an unexpected error in front of the room.
- ☐ Understand Streamlit's core execution model yourself, cold, before explaining it: **the entire script re-runs, top to bottom, every time a user interacts with any widget**. This single fact explains almost everything about how the app behaves, and it's worth having a clear, rehearsed way to state it (Part 1 covers this explicitly).
- ☐ Confirm `streamlit run app.py` opens correctly in a browser on your demo machine, and know the port (`localhost:8501` by default) in case a student's browser doesn't open automatically and needs the URL typed in manually.
- ☐ Know where the FastAPI/REST API lecture fits in your section's actual class schedule today, and communicate that timing to students at the start, so nobody is confused about why the day feels longer than a normal 75-minute lab.

4 Materials Needed

- Terminal (zsh), VS Code, Python 3.10+, the existing `module07_final_project` venv with a working `database.py` from Week 14
- A web browser for viewing the running Streamlit app

5 Timing Plan (75 minutes, lab portion only)

Time	Segment	Minutes
0:00–0:04	Welcome: “the interface, wired to last week’s data layer”	4
0:04–0:12	Part 1 — Install Streamlit and first run	8
0:12–0:36	Part 2 — Build the complete five-tab <code>app.py</code>	24
0:36–0:52	Part 3 — Test every feature	16

Time	Segment	Minutes
0:52–1:02	Part 4 — Adapt to your capstone domain	10
1:02–1:08	Part 5 — Ritual and push	6
1:08–1:15	Stretch preview + wrap-up, submission checklist	7

Part 2 receives the most time of the whole lab — five tabs’ worth of new widget syntax, each wired to a different database function, is genuinely the bulk of today’s new content; Part 3’s guided testing pass is also given real time, since it’s this lab’s actual verification step (there’s no `pytest` for UI behavior today).

6 Segment-by-Segment Walkthrough

6.1 Intro (0:00–0:04)

Say to the class:

*“Every function in last week’s `database.py` gets a real, clickable front end today. No terminal, no `sqlite3` shell, no Python REPL — a genuine web interface anyone could use. One fact about Streamlit explains almost everything you’ll see today, so I want to say it now, before any code: **Streamlit re-runs your entire script, top to bottom, every single time you interact with anything** — click a button, change a dropdown, type in a box. That’s not a bug or something to work around — it’s the whole model, and it’s why today’s code looks the way it does.”*

6.2 Part 1 — Install Streamlit and First Run (0:04–0:12, 8 min)

Teaching goal: Install Streamlit, run a minimal “hello world,” and see the re-run model in action for the first time, concretely.

Say to the class:

“Smallest possible Streamlit app first — two lines — just to confirm the pipeline works before building anything real.”

Live-code this:

```
cd ~/ism3232/module07_final_project
source .venv/bin/activate
pip install streamlit
pip freeze > requirements.txt
touch app.py
code app.py
```

```
import streamlit as st
st.title('Hello from Streamlit!')
st.write('Database connected.')
```

Run it:

```
streamlit run app.py
```

Line-by-line explanation:

- `import streamlit as st` — same aliasing convention as `pandas as pd` — `st` is the near-universal community shorthand.
- `st.title('...')`, `st.write('...')` — say explicitly, these two lines are not `print()` —

Streamlit functions like these don't send text to a terminal, they render actual formatted HTML elements in a browser tab.

- `streamlit run app.py` — **not `python3 app.py`** — worth flagging explicitly, since it's a genuinely different way of running a Python file than every prior week: `streamlit run` starts a small local web server and opens a browser tab pointed at it (`http://localhost:8501` by default), rather than just executing the script and printing to the terminal.

Verify: a browser tab opens showing “Hello from Streamlit!” as a large title and “Database connected.” as body text.

Common student mistakes to watch for:

- Running `python3 app.py` out of habit — this technically runs without error (it just executes the script as plain Python), but produces **no visible output at all**, since `st.title()/st.write()` need Streamlit's own server process to actually render anything; a good, low-stakes “why did nothing happen” moment worth resolving explicitly.
- Forgetting `pip install streamlit` before running — `ModuleNotFoundError: No module named 'streamlit'`, the same familiar missing-import error pattern from every prior module.

Check for understanding: “If you closed the browser tab but left the terminal command running, would the app still be ‘running’?” (Yes — the local server process (started by `streamlit run`) keeps running until you stop it in the terminal, typically with `Ctrl+C`; the browser tab is just a *view* onto that running server, and reopening `localhost:8501` in a new tab would show the same live app again. Worth stating explicitly, since Part 5 requires stopping the server before running the ritual.)

6.3 Part 2 — Build the Complete app.py (0:12–0:36, 24 min)

Teaching goal: Five tabs, each wired to a different database.py function — the lab’s central content, worth walking tab by tab, unhurried.

Say to the class:

“Five tabs, one file, each one calling a function you already built and tested last week. I want you to notice, tab by tab, which specific database.py function each one calls — that mapping is the entire structure of this file.”

Live-code this, replacing app.py’s hello-world content entirely:

```
import streamlit as st
from database import create_table, add_record, get_all_records, update_status, get_status_re

create_table()

st.set_page_config(page_title='Request Tracker', layout='wide')
st.title('Business Request Tracker')

tab1, tab2, tab3, tab4, tab5 = st.tabs(['Submit', 'View', 'Filter', 'Update', 'Report'])
```

Line-by-line explanation of the setup:

- `create_table()` — called once, at the very top of the script, with no `db_file` argument, so it uses the real `DB_FILE` default — say explicitly, this is a genuinely important detail worth stating: **because the entire script re-runs on every interaction (Part 1’s core fact), `create_table()` runs again every single time.** This is only safe because Week 14’s function used `CREATE TABLE IF NOT EXISTS` — worth connecting explicitly back to that specific design decision, made a full week earlier, paying off directly here.
- `st.set_page_config(page_title='Request Tracker', layout='wide')` — configures browser-tab title and page width; `layout='wide'` uses the full browser width instead of a narrower centered column, worth using for a data-heavy app like this one.
- `tab1, tab2, tab3, tab4, tab5 = st.tabs(...)` — **tuple unpacking, again, from Weeks 1/7/10** — `st.tabs()` returns five tab objects at once, matched by position to the five string labels given.

Tab 1 — Submit:

```
with tab1:
    st.header('Submit a New Request')
    requester = st.text_input('Your name')
    category = st.selectbox('Category', ['Travel', 'Equipment', 'Software', 'Other'])
```

```

amount      = st.number_input('Amount ($)', min_value=0.0, step=10.0)
notes       = st.text_area('Notes (optional)')
if st.button('Submit Request'):
    if requester and amount > 0:
        add_record(requester, category, amount, notes)
        st.success(f'Request submitted for {requester}')
    else:
        st.error('Name and amount are required.')

```

Line-by-line explanation:

- `with tab1:` — a `with` block, exactly Module 12’s file-handling pattern, here scoping which widgets belong to *this specific tab*, not the other four.
- `requester = st.text_input('Your name')` — a text box; `requester` holds whatever’s currently typed, **re-read fresh on every single re-run** — say explicitly, this is worth stating precisely: `st.text_input` doesn’t just capture a value once when submitted; on every re-run of the whole script (Part 1’s core model), this line runs again and picks up the widget’s *current* value.
- `st.selectbox('Category', [...])`, `st.number_input('Amount ($)', min_value=0.0, step=10.0)`, `st.text_area(...)` — a dropdown, a numeric spinner (`min_value=0.0` prevents negative amounts; `step=10.0` sets the increment/decrement size), and a multi-line text box — four different input widget types, one line each.
- `if st.button('Submit Request')`: — **this is the moment worth explaining most carefully, tying directly back to the re-run model.** `st.button(...)` returns `True` only on the exact re-run triggered by clicking it, and `False` on every other re-run (including the very next one, immediately after) — say explicitly: **the button’s `True` value is not “sticky”** — it doesn’t stay `True` after being clicked; it’s `True` for exactly one script execution, then reverts to `False`. This is why the code inside this `if` block — the actual `add_record()` call — only runs once per click, not continuously.
- `if requester and amount > 0:` — a validation check *inside* the button-click block — say explicitly, this combines Week 5’s truthiness ideas (`requester` alone is `True` if it’s a non-empty string, `False` if empty) with Week 5’s comparison operators, gating whether `add_record()` actually runs.
- `add_record(requester, category, amount, notes)` — **this is the entire wiring** — the UI collects four values from widgets, and hands them directly to Week 14’s function, completely unchanged from how it was called in `test_script.py` last week.
- `st.success(...)` / `st.error(...)` — Streamlit’s styled feedback messages (green/red boxes) — worth a brief note that these are purely presentational, no different in principle from Week 7’s `print()` calls in `main.py`, just rendered as styled UI elements instead of terminal text.

Tab 2 — View:


```

with tab2:
    st.header('All Requests')
    records = get_all_records()
    if records:
        st.dataframe(records, use_container_width=True)
    else:
        st.info('No records yet.')

```

Line-by-line explanation:

- `records = get_all_records()` — the exact same function call from every `test_script.py`/REPL test last week, now called fresh on every re-run, always reflecting the database's current contents.
- `st.dataframe(records, use_container_width=True)` — **Streamlit accepts a list of dictionaries directly** and renders it as an interactive, sortable table — say explicitly, this is precisely the `[dict(row) for row in ...]` shape `get_all_records()` was specifically designed to return back in Week 14; no conversion or reshaping is needed between the data layer and this display call.
- `use_container_width=True` — **the version-specific detail flagged in this guide's setup checklist** — this parameter stretches the table to the tab's full width. Verified in a current Streamlit release: this still works but emits a deprecation warning recommending `width='stretch'` instead. If your installed version has removed it entirely, replace every occurrence in this file with `width='stretch'`.
- `if records: ... else: st.info(...)` — a graceful empty-state message rather than showing a blank or broken table when the database has no records yet — worth noting as a small but genuinely good UX habit.

Tab 3 — Filter:

```

with tab3:
    st.header('Filter Requests')
    status_filter = st.selectbox('Filter by status', ['All', 'Pending', 'Approved', 'Rejected'])
    records = get_all_records()
    if status_filter != 'All':
        records = [r for r in records if r['status'] == status_filter]
    st.dataframe(records, use_container_width=True)
    st.caption(f'{len(records)} record(s) shown')

```

Line-by-line explanation:

- `status_filter = st.selectbox(...)` — a second, independent dropdown from Tab 1's category selector — say explicitly, each widget across the whole file is independent; there's no accidental cross-talk between Tab 1's category and Tab 3's `status_filter`, even though

both are built with `st.selectbox`.

- `if status_filter != 'All': records = [r for r in records if r['status'] == status_filter]` — **Week 6’s exact list-comprehension filter pattern**, applied to live database results instead of a hardcoded practice list. The 'All' option deliberately skips filtering entirely — worth noting this as a genuinely common, reusable UI pattern: an explicit “no filter” option alongside the real filter values.
- `st.caption(f'{len(records)} record(s) shown')` — small, muted helper text confirming the filtered count — a good, low-effort addition to any filtered view.

Tab 4 — Update:

```
with tab4:
    st.header('Update Request Status')
    records = get_all_records()
    if records:
        options = {f"ID {r['id']}: {r['requester']} - ${r['amount']:,.2f}": r['id'] for r in records}
        selected = st.selectbox('Select request', list(options.keys()))
        new_status = st.selectbox('New status', ['Pending', 'Approved', 'Rejected'])
        if st.button('Update Status'):
            update_status(options[selected], new_status)
            st.success(f'Updated to {new_status}')
    else:
        st.info('No records yet.')
```

Line-by-line explanation — this is the trickiest single line in the whole file, worth slowing down for:

- `options = {f"ID {r['id']}: ..." : r['id'] for r in records}` — a **dictionary comprehension** (new syntax, though the *shape* is Week 6/11’s familiar comprehension pattern): for every record, build a human-readable label string as the **key**, and that record’s actual id as the **value**. Say explicitly why: `st.selectbox` needs to display something *readable* to a human (a name and amount, not a bare database ID), but the code needs the actual id back to call `update_status(...)` correctly — this dictionary is the bridge between “what the user sees” and “what the database needs.”
- `selected = st.selectbox('Select request', list(options.keys()))` — the dropdown shows the *readable labels* (the dictionary’s keys); `selected` ends up holding whichever full label string the user picked.
- **Worth flagging explicitly, echoing Week 14’s ordering lesson:** since `options` is built by iterating `get_all_records()` — which orders by `id DESC` — the dropdown’s *first, default-selected* option is the **most recently added** request, not the first one ever submitted. This is a direct continuation of last week’s “read the ordering carefully” lesson, now showing up as a real UI behavior, not just a data-layer detail.
- `if st.button('Update Status'): update_status(options[selected], new_status)`

— `options[selected]` looks up the actual `id` corresponding to whichever label was chosen, and passes *that* into `update_status(...)` — exactly Week 14’s function signature, unchanged.

Tab 5 — Report:

```
with tab5:
    st.header('Status Report')
    report = get_status_report()
    if report:
        for row in report:
            col1, col2 = st.columns(2)
            col1.metric(row[0], row[1], 'requests')
            col2.metric('Total', f'${row[2]:,.2f}')
    else:
        st.info('No data yet.')
```

Line-by-line explanation:

- `report = get_status_report()` — Week 14’s aggregate function, returning plain tuples `((status, count, total))`, exactly as designed — say explicitly, this is why `row[0]`, `row[1]`, `row[2]` positional access appears here, matching the deliberate choice made last week not to use `row_factory` for this specific function.
- `col1, col2 = st.columns(2)` — **inside the loop**, creating a fresh two-column layout **for every status row** — say explicitly this is worth noting: each status gets its own pair of side-by-side metric boxes, not one shared pair reused across all statuses.
- `col1.metric(row[0], row[1], 'requests')` — `st.metric()` renders a large, styled number display; the three arguments are a label (the status name), the main value (the count), and a small caption/delta text ('requests', used here as a unit label rather than its more typical use showing a change/delta).
- `col2.metric('Total', f'${row[2]:,.2f}')` — a second metric box, same row, showing that status’s total dollar amount.

Run it:

`streamlit run app.py`

Common student mistakes to watch for:

- Placing widgets or function calls **outside** the correct `with tabN:` block — Streamlit doesn’t raise an error for this, it just renders the misplaced element in the wrong tab (or, if entirely outside any `with tab:` block, above/below the tabs entirely) — a good visual “does this look right” check, not something Python itself will catch.
- Forgetting that `import pandas as pd` is **not** needed for Tab 2/3’s `st.dataframe(records, ...)` — passing a plain list of dicts works directly; pandas only becomes necessary for the

Stretch section's chart.

- Confusing Tab 1's category variable name with Tab 4's options/selected — these are separate variables in separate with blocks; there's no naming collision risk here since each with tab: block's variables are just ordinary Python names in the same overall script scope, but genuinely distinct in purpose.

Check for understanding: “If a user clicks ‘Submit Request’ in Tab 1, does anything happen to what's currently displayed in Tab 2, without switching tabs or clicking anything else?” (Yes — since clicking any button triggers a full script re-run top to bottom, Tab 2's `get_all_records()` call also re-runs and picks up the newly added record, even though the user is still looking at Tab 1; if they switch to Tab 2, the new record will already be there. A good, concrete illustration of the whole-script re-run model's real consequence.)

6.4 Part 3 — Test Every Feature (0:36–0:52, 16 min)

Teaching goal: A guided, end-to-end test pass through the running application — this lab’s actual verification step, since there’s no pytest suite for UI behavior today.

Say to the class:

“No automated tests today — the interface itself is what we’re verifying, by actually using it, deliberately, tab by tab.”

Walk through each tab, live, having students follow along on their own running app:

Tab 1 — Submit: Add three test records with genuinely different amounts — one under \$500, one between \$500–\$2000, one over \$2000 (this range matters directly for Part 4’s domain-specific thresholds, worth noting explicitly). Confirm the green success message appears after each submission.

Tab 2 — View: Switch tabs and confirm all three records appear in the table — verified via direct simulation: after three submissions (Taylor \$1,200, Jordan \$450, Morgan \$3,500), the view table shows exactly three rows.

Tab 3 — Filter: Filter by ‘Pending’ — confirm the shown count matches (all three, if none have been updated yet — verified: 3 record(s) shown). Filter by ‘All’ to restore the full view.

Tab 4 — Update: Select the first (default) option in the dropdown — **remind the room explicitly this is the most recently submitted record, per Part 2’s ordering note, not necessarily the first one they typed** — and change its status to ‘Approved’. Switch to Tab 3, filter by ‘Approved’, and confirm exactly one record now appears there.

Tab 5 — Report: Confirm the metrics show correct counts and totals — verified, continuing the three-record example: Approved: 1, Total \$3,500.00 and Pending: 2, Total \$1,650.00 (Taylor’s \$1,200 + Jordan’s \$450).

Common student mistakes to watch for:

- Testing tabs in isolation, refreshing the browser between each, and losing track of what data is actually in the database at each step — encourage testing in one continuous session, in the stated order, so each tab’s expected state genuinely follows from the previous tab’s actions.
- Being confused when Tab 4’s default-selected dropdown option doesn’t match “the first thing I submitted” — this is the exact ordering behavior flagged in Part 2; use it as a live, real instance of the lesson rather than treating it as a bug to fix.

Check for understanding: “If you added a fourth record in Tab 1 right now, without touching any other tab, and then immediately checked Tab 5’s Report — would the totals reflect it?” (Yes — the moment the button click triggers a re-run, every tab’s code re-executes fresh, including Tab 5’s `get_status_report()` call, even though Tab 5 wasn’t the tab being interacted with. A good final reinforcement of the whole-script re-run model.)

6.5 Part 4 — Adapt to Your Capstone Domain (0:52–1:02, 10 min)

Teaching goal: Customize the generic five-tab template to match each student's own Week 13 proposal — genuinely making it *their* capstone, not a shared demo app with a different title.

Say to the class:

“Four specific things to change, matching your own proposal from three weeks ago — not a rewrite, targeted edits.”

State the four required changes explicitly, matching the lab page:

- `st.title()` — change to the student's own system name
- Category selectbox options — change to the student's own domain's categories
- Column names in the dataframe — must match the student's own database schema (from Week 13's `schema.sql` / Week 14's `create_table()`)
- Status options — change if the student's own system uses different status values than Pending/Approved/Rejected

Facilitation notes, since each student's actual edits will differ:

- **The most common mismatch to watch for:** a student who adapted `database.py`'s column names in Week 14 but forgot one of today's dropdown/label strings still references the old generic names (`requester`, `category`, `amount`) — walk the room specifically cross-checking today's `app.py` against each student's own `database.py` `CREATE TABLE` statement, field by field.
- **Encourage testing the customized version with the exact same Part 3 workflow** — submit a few records, view, filter, update, check the report — rather than assuming the customization is correct just because it runs without a Python error; a working script and a *correctly wired* script are different claims, and only re-running the full test pass confirms the second one.

Common student mistakes to watch for:

- Changing the displayed category *labels* in the selectbox but not correspondingly updating anything in `database.py` that might depend on specific category strings (if a student's own system has category-specific business logic) — worth a reminder that today's changes are UI-layer only; any actual logic differences belong back in `database.py`, following Week 14's patterns.
- Leaving `st.title('Business Request Tracker')` unchanged — a small thing, but worth catching, since a generic, un-customized title is an easy, visible signal a submission wasn't genuinely adapted.

6.6 Part 5 — Ritual and Push (1:02–1:08, 6 min)

Teaching goal: Stop the running Streamlit server before running the ritual — a genuinely new step this week — then the now-familiar format/lint/commit/push sequence.

Say to the class:

“One new step before the usual ritual: stop Streamlit first. It’s still running as a live server in your terminal — the ritual needs that terminal back.”

Do:

In the terminal running Streamlit:
Ctrl+C

Then the ritual — note no pytest this week, since there’s no test suite for the UI:

```
ruff format . && ruff check .  
git add . && git commit -m 'lab 15: Streamlit five-feature interface' && git push
```

Common student mistakes to watch for:

- Opening a *second* terminal tab to run the ritual while Streamlit is still running in the first, rather than stopping it — this actually works fine functionally (the two processes don’t conflict), but the lab page’s own instruction is to stop it first, and it’s a good habit regardless: a stopped, deliberate state is easier to reason about than “several things running at once, hopefully not interfering.”
- Committing `data/requests.db` despite Week 13’s `*.db .gitignore` entry — same check as every prior week: `git status` before `add/commit` catches an accidentally-tracked database file.

6.7 Stretch — Bar Chart in the Report Tab (1:08–1:12, as time allows)

Frame as a quick, genuinely nice visual upgrade if the room reaches it:

```
# In Tab 5, after getting the report:
import pandas as pd
df = pd.DataFrame(report, columns=['Status', 'Count', 'Total'])
st.bar_chart(df.set_index('Status')['Total'])
```

One sentence of framing, if you demo this: “report is a list of plain tuples — `pd.DataFrame(report, columns=...)` wraps it in a real DataFrame, exactly Unit 4’s Module 13 skills from ISM2411-adjacent pandas work if your students have that background, or a light first taste if not. `.set_index('Status')['Total']` picks out just the `Total` column, indexed by status name, which is exactly the shape `st.bar_chart()` expects — one bar per status, height equal to that status’s total revenue.” Verified: replaces the metric boxes with a two-bar chart (Approved: \$3,500, Pending: \$1,650) for the guide’s running example data.

7 Wrap-Up (last ~3 minutes)

Review the submission checklist together:

- ☐ `app.py` contains all five tabs, each correctly wired to its `database.py` function
- ☐ `st.title()`, category options, column names, and status options all customized to the student's own capstone domain
- ☐ All five tabs tested end to end with real submitted data (Part 3's workflow)
- ☐ Git commit made, with a message including "lab 15"
- ☐ Full ritual run (Streamlit stopped, format, lint, commit, push)
- ☐ GitHub repository URL pasted into Canvas

Preview Week 16: "One feature remains from your Week 13 proposal: the GenAI feature. Next week wires an AI call into your interface, and is also the final capstone demo — everything from Week 1's % prompt check through today's Streamlit tabs comes together as one finished, presentable system."

8 Appendix A — Full Answer Key (`app.py`)

```
# app.py
import streamlit as st
from database import create_table, add_record, get_all_records, update_status, get_status_re

create_table()

st.set_page_config(page_title='Request Tracker', layout='wide')
st.title('Business Request Tracker')

tab1, tab2, tab3, tab4, tab5 = st.tabs(['Submit', 'View', 'Filter', 'Update', 'Report'])

# --- Tab 1: Add a record ---
with tab1:
    st.header('Submit a New Request')
    requester = st.text_input('Your name')
    category = st.selectbox('Category', ['Travel', 'Equipment', 'Software', 'Other'])
    amount = st.number_input('Amount ($)', min_value=0.0, step=10.0)
    notes = st.text_area('Notes (optional)')
    if st.button('Submit Request'):
        if requester and amount > 0:
            add_record(requester, category, amount, notes)
            st.success(f'Request submitted for {requester}')
        else:
```

```

        st.error('Name and amount are required.')

# --- Tab 2: View all records ---
with tab2:
    st.header('All Requests')
    records = get_all_records()
    if records:
        st.dataframe(records, use_container_width=True)
    else:
        st.info('No records yet.')

# --- Tab 3: Filter records ---
with tab3:
    st.header('Filter Requests')
    status_filter = st.selectbox('Filter by status', ['All', 'Pending', 'Approved', 'Rejected'])
    records = get_all_records()
    if status_filter != 'All':
        records = [r for r in records if r['status'] == status_filter]
    st.dataframe(records, use_container_width=True)
    st.caption(f'{len(records)} record(s) shown')

# --- Tab 4: Update status ---
with tab4:
    st.header('Update Request Status')
    records = get_all_records()
    if records:
        options = {f"ID {r['id']}: {r['requester']} - ${r['amount']:,.2f}": r['id'] for r in records}
        selected = st.selectbox('Select request', list(options.keys()))
        new_status = st.selectbox('New status', ['Pending', 'Approved', 'Rejected'])
        if st.button('Update Status'):
            update_status(options[selected], new_status)
            st.success(f'Updated to {new_status}')
    else:
        st.info('No records yet.')

# --- Tab 5: Status report ---
with tab5:
    st.header('Status Report')
    report = get_status_report()
    if report:
        for row in report:

```

```

        col1, col2 = st.columns(2)
        col1.metric(row[0], row[1], 'requests')
        col2.metric('Total', f'${row[2]:,.2f}')
    else:
        st.info('No data yet.')

```

Verified end-to-end behavior (via `streamlit.testing.v1.AppTest`, simulating real widget interactions rather than just reading the code): submitting Taylor (\$1,200), Jordan (\$450), and Morgan (\$3,500) via Tab 1, then approving the default-selected (most recent, per the `id DESC` ordering) record in Tab 4, produces exactly this Tab 5 report:

```

Approved: 1 requests | Total: $3,500.00
Pending:  2 requests | Total: $1,650.00

```

Stretch (bar chart):

```

import pandas as pd
df = pd.DataFrame(report, columns=['Status', 'Count', 'Total'])
st.bar_chart(df.set_index('Status')['Total'])

```

9 Appendix B — Extra Practice (only if the class finishes early)

Five parts fill the full class period at a normal pace, especially given Part 3's deliberate end-to-end testing depth. If a section moves unusually fast:

Extra — a delete button in Tab 4. Using Appendix B's `delete_record` function from Week 14's guide, add a second button to Tab 4: `if st.button('Delete Request'):` `delete_record(options[selected]); st.warning(f'Deleted request {selected}')`. Discuss briefly: should a delete action require a confirmation step in a real system, given it's permanent? (No required answer — a good design discussion connecting back to Week 2's `rm` safety-ritual theme, now in a GUI context instead of a terminal.)

Extra — a search box in Tab 2. Add `search = st.text_input('Search by name')` above the Tab 2 table, and filter records to only those whose requester contains the search text (case-insensitive): `records = [r for r in records if search.lower() in r['requester'].lower()]` — a good extra rep of the list-comprehension-filter pattern, on a genuinely different kind of condition (substring match) than Tab 3's exact-match status filter.